第 4 章：序列类型

本章目标：不只会写列表和元组，还要能判断一次操作究竟是“读取原对象”“创建新对象”，还是“原地修改对象”。配套代码：`example.py` / `Chapter04Sequences.java`。

建议先阅读本章，再直接运行：

```
python java_to_python/chapters/04_sequences/example.py
```

示例输出按照 4.1～4.8 分段。学习时先猜结果，再修改索引、切片边界和示例数据重跑。

学习目标

学完本章后，你应该能够：

- 用统一的“有顺序的一排位置”理解列表、元组和字符串；
- 正确使用正索引、负索引和半开区间切片；
- 区分 `append` 与 `extend`、`sort` 与 `sorted`、`+` 与 `+=`；
- 解释浅拷贝为什么不能隔离嵌套的可变对象；
- 使用元组打包、解包多个值，并理解“元组不可变”的准确含义；
- 根据数据是否需要修改、元素含义是否固定来选择 `list`、`tuple` 或 `str`。

4.1 什么是序列

序列可以先想成“一排有固定先后次序的位置”。每个位置都有从 `0` 开始的索引，因此通常可以进行以下操作：

意图	Python	结果
取得长度	`len(values)`	一个整数
取单个元素	`values[index]`	原序列中的一个对象
取一段	`values[start:stop]`	一个新序列

判断是否包含值	`target in values`	`True` 或 `False`
连接同类序列	`left + right`	一个新序列
重复内容	`values * count`	一个新序列

本章的三个核心序列类型各有不同用途：

类型	有序	可修改	常见用途	Java 心智模型
`list`	是	是	数量可变的一组同类数据	`ArrayList<T>`
`tuple`	是	否	固定结构的小记录、多个返回值	record/小型值对象
`str`	是	否	文本	`String`

“可修改”指对象创建后能否原地改变。变量仍然可以重新绑定，例如 `text = text.upper()` 合法，但它让 `text` 指向了一个新字符串，并没有修改旧字符串。

4.2 列表：有序且可变

创建列表

方括号创建列表字面量，`list(iterable)` 把其他可迭代对象转换为列表：

```
languages: list[str] = ["Java", "Python", "Go"]
letters = list("API")

print(languages)  # ['Java', 'Python', 'Go']
print(letters)    # ['A', 'P', 'I']
```

和 `ArrayList<Object>` 类似，一个 Python 列表在运行时可以混装不同类型。实际业务代码通常让元素保持同一种含义，并用 `list[str]` 这样的类型提示帮助 IDE 发现错误。类型提示默认不会在运行时强制检查。

正索引、负索引与越界

正索引从左向右计数，负索引从右向左计数：

```
值:      Java    Python    Go    Rust
```

正索引、负索引与越界

正索引从左向右计数，负索引从右向左计数：

```
值:       Java    Python    Go    Rust
正索引:     0        1       2      3
负索引:    -4       -3      -2     -1
```

```
languages = ["Java", "Python", "Go", "Rust"]
print(languages[0])   # Java
print(languages[-1])  # Rust
```

索引必须真正指向一个元素。空列表没有 `[-1]`，过大的索引也不会自动返回 `None`，而会抛出 `IndexError`：

```
empty: list[str] = []
# print(empty[-1])  # IndexError: list index out of range
```

如果序列可能为空，应先明确业务规则：空时跳过、提供默认值，还是主动报错。不要为了避免异常而随意吞掉数据问题。

4.3 切片：用半开区间取得一段

完整语法是 `values[start:stop:step]`：

- `start`：从哪个索引开始，包含该位置；
- `stop`：到哪个索引停止，不包含该位置；
- `step`：每次跨多少个位置，默认是 `1`，不能是 `0`。

`start:stop` 采用左闭右开的半开区间 `[start, stop)`，因此 `values[1:4]` 最多包含索引 `1`、`2`、`3`。长度恰好是 `stop - start`，也方便让相邻切片无缝衔接：

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers[1:4])   # [20, 30, 40]
print(numbers[:3])    # [10, 20, 30]
print(numbers[3:])    # [40, 50, 60]
print(numbers[::2])   # [10, 30, 50]
print(numbers[::-1])  # [60, 50, 40, 30, 20, 10]
```

iz72502x6dg6rpjj0sn140000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

切片边界超出范围时会被收缩到有效范围，通常不会像单个索引那样抛出 `IndexError`：

```
print(numbers[:100])  # 返回全部元素
print(numbers[100:])  # 返回空列表
```

这是一个需要主动记住的差异：**单个索引要求位置存在，切片描述的是一个范围。**

切片产生浅拷贝

对列表做普通切片会创建一个新的外层列表：

```
original = ["Java", "Python"]
copied = original[:]

print(original == copied)  # True：内容相等
print(original is copied)  # False：不是同一个列表对象
```

但这只是浅拷贝（shallow copy）：复制的是外层容器中的引用，内层可变对象仍然共享。

```
original = [["Java"], ["Python"]]
copied = original[:]
copied[0].append("Kotlin")

print(original)                 # [['Java', 'Kotlin'], ['Python']]
print(original[0] is copied[0])  # True
```

如果只会替换、增删外层元素，浅拷贝通常足够；如果还要独立修改嵌套内容，就应重新构造每个内层对象，或在确实需要复制任意复杂对象图时谨慎使用 `copy.deepcopy`。

4.4 修改列表

列表方法大致分为“修改当前列表”和“计算并返回新对象”两类。先判断自己是否允许调用者手中的列表一起变化，再选择 API。

`append`、`extend` 与 `insert`

```
skills = ["Java"]
```

7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

吣... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

4.4 修改列表

列表方法大致分为“修改当前列表”和“计算并返回新对象”两类。先判断自己是否允许调用者手中的列表一起变化，再选择 API。

`append`、`extend` 与 `insert`

```
skills = ["Java"]
skills.append("Python")          # 添加一个元素
skills.extend(["SQL", "Git"])  # 从可迭代对象中逐个添加
skills.insert(1, "Linux")        # 在索引 1 前插入
```

`append(value)` 永远把 `value` 当成一个整体。`extend(iterable)` 会遍历实参，再逐个添加：

```
items = ["Java"]
items.append(["SQL", "Git"])
print(items)  # ['Java', ['SQL', 'Git']]

letters = ["Java"]
letters.extend("SQL")
print(letters)  # ['Java', 'S', 'Q', 'L']，字符串也可迭代
```

想加入一个字符串时用 `append("SQL")`；想合并一组字符串时用 `extend(["SQL", "Git"])`。

删除元素

写法	按什么删除	返回值或错误
`pop()`	最后一个位置	返回被删除的元素；空列表抛 `IndexError`
`pop(index)`	指定位置	返回被删除的元素；越界抛 `IndexError`
`remove(value)`	第一个相等的值	返回 `None`；找不到抛 `ValueError`
`del values[index]`	位置或切片	删除后没有返回值
`clear()`	全部元素	原地清空并返回 `None`

需要继续使用被删元素时选 `pop`；已知值但不知道位置时选 `remove`；只关心删除位置时用 `del`。如果目标值可能不存在，应先决定“不存在”究竟是正常分支还是数据错误。

/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家

需要继续使用被删元素时选 `pop`；已知值但不知道位置时选 `remove`；只关心删除位置时用 `del`。如果目标值可能不存在，应先决定"不存在"究竟是正常分支还是数据错误。

原地方法通常返回 `None`

许多列表方法遵循同一设计：明确修改当前对象，并返回 `None`，避免让人误以为得到了新列表。

```
numbers = [3, 1, 2]
result = numbers.sort()

print(numbers)  # [1, 2, 3]
print(result)  # None
```

常见错误是写成 `numbers = numbers.sort()`，这会让 `numbers` 重新绑定到 `None`。

切片赋值

切片位于赋值号左边时，不再是读取副本，而是原地替换一个范围：

```
queue = ["A", "B", "C", "D"]
queue[1:3] = ["X", "Y", "Z"]
print(queue)  # ['A', 'X', 'Y', 'Z', 'D']
```

替换前后的元素数量可以不同。它很强大，但范围复杂时普通代码可能更容易读懂。

4.5 新列表与原地修改

`+` 与 `+=` 不是始终等价

`left + right` 创建新列表；列表的 `left += right` 通常原地扩展左侧列表：

```
first = [1, 2]
alias = first

first = first + [3]
print(first)  # [1, 2, 3]
print(alias)  # [1, 2], first 已指向新列表
```

```
first = [1, 2]
alias = first

first += [3]
print(first)  # [1, 2, 3]
print(alias)  # [1, 2, 3]，同一个列表被原地修改
```

第 3 章的“名字绑定”在这里直接影响程序行为：只要存在别名，是否创建新对象就不是内部细节。

`sort` 与 `sorted`

`list.sort()` 只适用于列表，原地排序并返回 `None`。`sorted(iterable)` 接收任意可迭代对象，返回一个新列表：

```
scores = [88, 95, 91]
ranking = sorted(scores, reverse=True)

print(scores)   # [88, 95, 91]
print(ranking)  # [95, 91, 88]
```

实际数据通常需要指定按哪个字段排序。`key` 函数会接收每个元素，并返回真正参与比较的值：

```
students = [("Alice", 88), ("Bob", 95), ("Cara", 91)]
ranking = sorted(students, key=lambda student: student[1], reverse=True)
print(ranking)  # [('Bob', 95), ('Cara', 91), ('Alice', 88)]
```

这里的 `lambda` 是一个小型匿名函数，相当于“取出学生元组的第 2 项”。函数会在第 7 章详细学习；本章先把它当作排序规则。若规则复杂，应改用有名字的函数。

`*` 与嵌套列表共享

一维列表 `[0] * 4` 没问题，因为共享不可变整数不会造成联动修改。二维列表则不同：

```
wrong = [[0] * 3] * 3
wrong[0][0] = 1
print(wrong)  # 三行都变成 [1, 0, 0]
```

`* 3` 复制了三个内层列表引用，并没有创建三个内层列表。可把它想成给同一块白板挂了三个

/f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

`*` 与嵌套列表共享

一维列表 `[0] * 4` 没问题，因为共享不可变整数不会造成联动修改。二维列表则不同：

```
wrong = [[0] * 3] * 3
wrong[0][0] = 1
print(wrong)  # 三行都变成 [1, 0, 0]
```

`* 3` 复制了三个内层列表引用，并没有创建三个内层列表。可把它想成给同一块白板挂了三个门牌，从任何一个门牌进入，修改的都是同一块白板。

独立创建每一行要使用列表推导式：

```
correct = [[0 for _ in range(3)] for _ in range(3)]
correct[0][0] = 1
print(correct)  # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

外层每循环一次都会执行一次内层表达式，因而产生新的列表对象。`_` 只是普通变量名，这里表示循环值不会被使用。推导式会在第 6 章系统学习。

4.6 元组：固定槽位的不可变序列

元组常用于固定结构的小数据、解包和函数的多个返回值：

```
point = (120, 30)
x, y = point
print(x, y)  # 120 30
```

逗号创建元组

严格来说，创建元组的是逗号，不是圆括号：

```
singleton = (42,)
not_a_tuple = (42)

print(type(singleton).__name__)   # tuple
print(type(not_a_tuple).__name__) # int
```

vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

目关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家

空元组写作 `()`。实际代码仍推荐加圆括号，让边界更清楚。

打包、解包与星号接收

```
record = "ORD-001", "paid", 299  # 自动打包成元组
order_id, status, amount = record
first, *middle, last = [10, 20, 30, 40]

print(middle)  # [20, 30]，带星号的目标得到列表
```

普通解包要求左右数量一致，否则抛 `ValueError`。一个赋值语句中只能有一个带星号的接收目标，因为 Python 必须能唯一确定各部分如何绑定。

元组不可变，不代表内部对象都不可变

元组创建后不能替换、增加或删除槽位，但槽位指向的可变对象仍然能改变：

```
profile = ("Alice", ["Java"])
profile[1].append("Python")  # 合法：修改的是内部列表
# profile[1] = ["Go"]        # TypeError：试图替换元组槽位
```

因此“放进元组”不是深度冻结。长期维护的业务模型若字段较多，Java 类或 Python `dataclass` 往往比靠 `record[3]` 猜字段含义更清楚。

4.7 字符串也是不可变序列

字符串支持长度、索引、切片、成员判断和迭代：

```
text = "Python"
print(len(text))       # 6
print(text[0])         # P
print(text[1:4])       # yth
print(text[::-1])      # nohtyP
print("tho" in text)  # True
```

但不能通过索引修改字符：

n/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代_ 商家

但不能通过索引修改字符：

```
# text[0] = "J"  # TypeError: 'str' object does not support item assignment
```

`upper` 、 `replace` 、 `strip` 等字符串方法都会返回新字符串：

```
normalized = "  java,python  ".strip().replace("java", "Java")
print(normalized)  # Java,python
```

拼接少量固定文本可以用 `+` 。连接一批字符串时优先用分隔符的 `join` ：

```
skills = ["Java", "Python", "SQL"]
summary = " / ".join(skills)
print(summary)  # Java / Python / SQL
```

`join` 要求所有元素都是字符串；数字等其他对象需要先明确转换。这能避免语言偷偷决定格式。

4.8 f-string、格式规格与转义

f-string 会在运行到该语句时计算花括号中的表达式并生成新字符串：

```
name = "机械键盘"
price = 299.0
discount_rate = 0.15

print(f"{name}: ¥{price:,.2f}")        # 机械键盘: ¥299.00
print(f"折扣率: {discount_rate:.1%}")  # 折扣率: 15.0%
```

常用格式如下：

格式	含义	示例结果
`.2f`	固定两位小数	`19.90`
`,.2f`	千位分隔并保留两位小数	`12,345.60`
`.1%`	乘以 100 后显示百分号	`25.6%`
`>10` / `<10` / `^10`	右对齐/左对齐/居中，宽度 10	表格文本

rf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

常用格式如下：

格式	含义	示例结果
`.2f`	固定两位小数	`19.90`
`,.2f`	千位分隔并保留两位小数	`12,345.60`
`.1%`	乘以 100 后显示百分号	`25.6%`
`>10` / `<10` / `^10`	右对齐/左对齐/居中，宽度 10	表格文本
`!r`	使用适合调试的表示形式	字符串会带引号

日志调用通常仍优先使用参数化日志，让日志框架只在需要输出时格式化。

`\n`、`\t` 和 `\\` 分别表示换行、制表符和反斜杠。Windows 路径或正则表达式可以使用原始字符串减少转义：

```
path = r"C:\temp\new"
```

原始字符串不是“完全不处理任何字符”，它主要改变反斜杠的解释方式，并且不能以单个反斜杠结尾。下面是语法错误，因此只作为注释展示：

```
# invalid_path = r"C:\temp\"  # 末尾反斜杠会转义结束引号
```

常见错误速查

错误写法或假设	实际问题	建议
`values[len(values)]` 取最后一项	最后一个索引是 `len(values) - 1`	非空时用 `values[-1]`
认为 `values[1:3]` 包含索引 3	`stop` 不包含在结果中	把它读作 `[1, 3)`
`items = items.sort()`	`sort` 原地修改并返回 `None`	用 `items.sort()` 或 `items = sorted(items)`
`items.extend("SQL")`	会加入三个字符	加单项用 `append("SQL")`
`copied = nested[:]`	浅拷贝仍共享内层对象	逐层构造或按需深拷贝

/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

录相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

常见错误速查

错误写法或假设	实际问题	建议
`values[len(values)]` 取最后一项	最后一个索引是 `len(values) - 1`	非空时用 `values[-1]`
认为 `values[1:3]` 包含索引 3	`stop` 不包含在结果中	把它读作 `[1, 3)`
`items = items.sort()`	`sort` 原地修改并返回 `None`	用 `items.sort()` 或 `items = sorted(items)`
`items.extend("SQL")`	会加入三个字符	加单项用 `append("SQL")`
`copied = nested[:]` 后修改内层	浅拷贝仍共享内层对象	逐层构造或按需深拷贝
`matrix = [[0] * 3] * 3`	三行指向同一列表	用推导式逐行创建
`(42)` 是单元素元组	它只是整数 42	写 `(42,)`
元组内所有对象都不可变	元组只禁止替换槽位	留意内部的列表、字典、集合

Java 心智切换

Java 常按具体类寻找 `get` 、 `subList` 、 `substring` 、 `Collections.reverse` 等方法；Python 让列表、元组和字符串共享索引与切片这套表达方式。迁移时最重要的不是代码变短，而是每次看见序列操作都问三个问题：

1. 结果是单个元素，还是一个新序列？
2. 原对象有没有被修改？
3. 新旧容器是否仍共享内部可变对象？

把这三个问题答清楚，切片、排序、 `+` 、 `+=` 、 `*` 和嵌套列表的大多数坑都会变得可预测。

小练习

练习 1：分页切片

ı/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-c3f63a6e440d1c091e1a262ab.html

求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

`(42)` 是单元素元组	它只是整数 42	写 `(42,)`
元组内所有对象都不可变	元组只禁止替换槽位	留意内部的列表、字典、集合

Java 心智切换

Java 常按具体类寻找 `get` 、 `subList` 、 `substring` 、 `Collections.reverse` 等方法；Python 让列表、元组和字符串共享索引与切片这套表达方式。迁移时最重要的不是代码变短，而是每次看见序列操作都问三个问题：

1. 结果是单个元素，还是一个新序列？
2. 原对象有没有被修改？
3. 新旧容器是否仍共享内部可变对象？

把这三个问题答清楚，切片、排序、 `+` 、 `+=` 、 `*` 和嵌套列表的大多数坑都会变得可预测。

小练习

练习 1：分页切片

给定 10 个订单编号和 `page_size = 3` ，只使用切片分别取出第 1、2、4 页。验证最后一页不足 3 项时不会报错，并思考页码从 1 开始时如何计算 `start` 与 `stop` 。

练习 2：安全更新技能

给定 `profile = ["Alice", ["Java", "SQL"]]` ，创建一个新结构并加入 `"Python"` ，要求原结构中的技能列表不变。分别尝试普通切片和手工复制内层列表，解释结果差异。

练习 3：成绩排名

给定 `[("Alice", 88), ("Bob", 95), ("Cara", 91)]` ，按分数降序产生新列表，原列表保持不变；然后使用解包分别取得冠军和其余选手。

练习 4：整理标签文本

给定字符串 `" java,python,sql "` ，生成 `"JAVA / PYTHON / SQL"` 。尝试组合 `strip` 、 `split` 、字符串方法和 `join` ，每一步打印中间结果，观察哪些步骤创建了新对象。

```python
"""第 4 章：列表、切片、元组与字符串序列。

这个文件把每个知识点拆成独立示例。直接运行后，可以对照输出观察：
一次操作是读取原对象、创建新对象，还是原地修改对象。
"""


def show_section(title: str) -> None:
    """为每组输出增加清晰边界。"""
    print(f"\n{'=' * 12} {title} {'=' * 12}")


def paginate(items: list[str], page_number: int, page_size: int) -> list[str]:
    """把从 1 开始的页码转换为半开切片边界。"""
    if page_number < 1 or page_size < 1:
        raise ValueError("页码和每页数量必须是正整数")
    start: int = (page_number - 1) * page_size
    stop: int = start + page_size
    return items[start:stop]


def demonstrate_indexing() -> None:
    """展示正负索引，以及单个索引和切片的越界差异。"""
    show_section("4.1~4.2 序列与列表索引")

    languages: list[str] = ["Java", "Python", "Go", "Rust"]
    print("完整列表:", languages)
    print("长度:", len(languages))
    print("首尾元素:", languages[0], languages[-1])
    print("倒数第二项:", languages[-2])
    print("是否包含 Python:", "Python" in languages)
    print("是否包含 Ruby:", "Ruby" in languages)

    # 教学反例：索引必须对应一个真实元素。这里只捕获明确的 IndexError，
    # 让后面的教学示例仍能继续运行；异常处理会在后续章节系统学习。
    try:
        print(languages[len(languages)])
```

```python
def demonstrate_indexing() -> None:

    # 教学反例：索引必须对应一个真实元素。这里只捕获明确的 IndexError，
    # 让后面的教学示例仍能继续运行；异常处理会在后续章节系统学习。
    try:
        print(languages[len(languages)])
    except IndexError as error:
        print("索引越界:", type(error).__name__, "-", error)

    empty: list[str] = []
    print("空列表切片是安全的:", empty[:1])


def demonstrate_slicing_and_copying() -> None:
    """展示半开切片、步长，以及浅拷贝的边界。"""
    show_section("4.3 切片与浅拷贝")

    numbers: list[int] = [10, 20, 30, 40, 50, 60]
    print("切片 [1:4]:", numbers[1:4])
    print("前三项 / 其余项:", numbers[:3], numbers[3:])
    print("每隔一项:", numbers[::2])
    print("反转副本:", numbers[::-1])
    print("超出边界会收缩:", numbers[:100], numbers[100:])

    copied: list[int] = numbers[:]
    print("刚复制时内容相等 / 对象相同:", numbers == copied, numbers is copied)
    copied.append(70)
    print("外层副本可独立增删:", copied)
    print("原列表保持不变:", numbers)

    # 浅拷贝只创建新的外层列表，里面的列表仍是同一个对象。
    nested: list[list[str]] = [["Java"], ["Python"]]
    shallow_copy: list[list[str]] = nested[:]
    shallow_copy[0].append("Kotlin")
    print("浅拷贝后修改内层:", nested, shallow_copy)
    print("内层对象仍共享:", nested[0] is shallow_copy[0])

```

```
45  def demonstrate_slicing_and_copying() -> None:
69      # 已知结构时，逐个复制内层列表往往比盲目深拷贝更明确。
70      independent_copy: list[list[str]] = [skills.copy() for skills in nested]
71      independent_copy[0].append("Go")
72      print("逐层复制后原数据:", nested)
73      print("逐层复制后的新数据:", independent_copy)
74
75
76  def demonstrate_list_mutation() -> None:
77      """对比添加、删除、切片赋值和原地方法的返回值。"""
78      show_section("4.4 修改列表")
79
80      skills: list[str] = ["Java"]
81      skills.append("Python")  # 把字符串作为一个完整元素加入。
82      skills.extend(["SQL", "Git"])  # 遍历右侧列表，逐个加入其中的元素。
83      skills.insert(1, "Linux")
84      print("append / extend / insert:", skills)
85
86      # 教学反例：字符串也是可迭代对象，extend 会逐个添加字符。
87      wrong_extend: list[str] = ["Java"]
88      wrong_extend.extend("SQL")
89      print("错误使用 extend 添加字符串:", wrong_extend)
90
91      removed_last: str = skills.pop()
92      skills.remove("SQL")
93      del skills[0]
94      print("删除后的列表:", skills, "pop 返回:", removed_last)
95
96      queue: list[str] = ["A", "B", "C", "D"]
97      queue[1:3] = ["X", "Y", "Z"]
98      print("切片赋值可改变长度:", queue)
99
100     numbers: list[int] = [3, 1, 2]
101     sort_result: None = numbers.sort()
102     print("sort 后的原列表:", numbers)
103     print("sort 的返回值:", sort_result)
```

```python
def demonstrate_new_object_or_in_place() -> None:
    """对比 +、+=、sort 和 sorted 对对象身份的影响。"""
    show_section("4.5 新对象与原地修改")

    added: list[int] = [1, 2]
    added_alias: list[int] = added
    added: list[int] = added + [3]  # 先创建新列表，再让 added 指向它。
    print("使用 + 后:", added, "旧别名:", added_alias)
    print("使用 + 后仍是同一对象:", added is added_alias)

    extended: list[int] = [1, 2]
    extended_alias: list[int] = extended
    extended += [3]  # list 的 += 原地扩展，别名也能看到变化。
    print("使用 += 后:", extended, "别名:", extended_alias)
    print("使用 += 后仍是同一对象:", extended is extended_alias)

    scores: list[int] = [88, 95, 91]
    ranking: list[int] = sorted(scores, reverse=True)
    print("sorted 产生的新列表:", ranking)
    print("原列表不变:", scores)

    students: list[tuple[str, int]] = [("Alice", 88), ("Bob", 95), ("Cara", 91), ("Dan", 91)]
    # key 对每个元组取索引 1，也就是分数；reverse=True 表示降序。
    student_ranking: list[tuple[str, int]] = sorted(students, key=lambda student: student[1], reverse=True)
    champion: tuple[str, int], *others = student_ranking
    print("学生排名:", student_ranking)
    print("冠军:", champion, "其余选手:", others)
    print("同分者保持原相对顺序:", student_ranking[1:3])


def demonstrate_repeated_nested_lists() -> None:
    """让二维列表的引用共享变得可观察。"""
    show_section("4.5 嵌套列表与 *")

    # 教学反例：* 只把同一个内层列表的引用复制了 3 份。
    shared_matrix: list[list[int]] = [[0] * 3] * 3
```

```python
def demonstrate_repeated_nested_lists() -> None:
    """让二维列表的引用共享变得可观察。"""
    show_section("4.5 嵌套列表与 *")

    # 教学反例：* 只把同一个内层列表的引用复制了 3 份。
    shared_matrix: list[list[int]] = [[0] * 3] * 3
    shared_matrix[0][0] = 1
    print("共享内层列表:", shared_matrix)
    print("各行是否和第 1 行相同:", [row is shared_matrix[0] for row in shared_matrix])
    print("第 1、2 行是同一对象:", shared_matrix[0] is shared_matrix[1])

    # 正确示范：外层每次循环都会执行 []，创建新的内层列表。
    independent_matrix: list[list[int]] = [[0 for _ in range(3)] for _ in range(3)]
    independent_matrix[0][0] = 1
    print("独立内层列表:", independent_matrix)
    same_as_first: list[bool] = [row is independent_matrix[0] for row in independent_matrix]
    print("各行是否和第 1 行相同:", same_as_first)
    print("第 1、2 行是同一对象:", independent_matrix[0] is independent_matrix[1])


def demonstrate_tuples() -> None:
    """展示元组创建、解包，以及元组内部可变对象。"""
    show_section("4.6 元组与解包")

    point = (120, 30)
    x: Literal[120], y: Literal[30] = point
    print("元组解包:", x, y)

    singleton = (42,)
    not_a_tuple = (42)
    print("单元素元组:", singleton, type(singleton).__name__)
    print("没有逗号:", not_a_tuple, type(not_a_tuple).__name__)

    order = "ORD-001", "paid", 299
    order_id: Literal['ORD-001'], status: Literal['paid'], amount: Lite… = order
    print("打包和解包:", order_id, status, amount)
```

```python
def demonstrate_tuples() -> None:

    first: int, *middle, last: int = [10, 20, 30, 40]
    print("扩展解包:", first, middle, last)

    # 元组槽位不能替换，但槽位所指向的列表仍然可以修改。
    profile: tuple[Literal['Alice'], list[str]] = ("Alice", ["Java"])
    profile[1].append("Python")
    print("元组中的可变列表:", profile)


def demonstrate_strings() -> None:
    """展示字符串的序列行为、不可变性、格式化和转义。"""
    show_section("4.7~4.8 字符串序列与格式化")

    text = "Python"
    print("字符串索引/切片:", text[0], text[1:4], text[::-1])
    print("子串成员判断:", "tho" in text)

    upper_text: LiteralString = text.upper()
    replaced_text: LiteralString = text.replace("Py", "Jy")
    print("字符串方法返回新对象:", upper_text, replaced_text)
    print("原字符串不变:", text)

    raw_tags = "  java,python,sql  "
    tags: list[LiteralString] = raw_tags.strip().upper().split(",")
    summary: LiteralString = " / ".join(tags)
    print("清理并连接标签:", summary)

    product = "机械键盘"
    price = 12_345.6
    discount_rate = 0.15
    print(f"格式化: {product} ¥{price:,.2f}, 折扣率 {discount_rate:.1%}")
    print(f"调试表示 !r 会显示引号: {text!r}")
    print("转义字符串: 第一行\n第二行\t制表符")
    print(r"原始字符串: C:\temp\new")

```

```python
def demonstrate_readme_exercises() -> None:
    """运行 README 中的分页、复制、排名和标签整理练习。"""
    show_section("README 小练习")

    print("练习 1：分页切片")
    order_ids: list[str] = [f"ORD-{number:03d}" for number in range(1, 11)]
    for page_number in (1, 2, 4):
        page: list[str] = paginate(order_ids, page_number, page_size=3)
        print(f"  第 {page_number} 页:", page)

    print("练习 2：安全更新嵌套技能")
    shallow_profile: list[Any] = ["Alice", ["Java", "SQL"]]
    shallow_updated: list[Any] = shallow_profile[:]
    shallow_skills: Any = shallow_updated[1]
    assert isinstance(shallow_skills, list)
    shallow_skills.append("Python")
    print("  普通切片仍共享技能列表:", shallow_profile)
    print("  内层身份相同:", shallow_profile[1] is shallow_updated[1])

    original_profile: list[Any] = ["Alice", ["Java", "SQL"]]
    original_skills: Any = original_profile[1]
    assert isinstance(original_skills, list)
    safe_skills: list[Any] = original_skills.copy()
    safe_profile: list[Any] = [original_profile[0], safe_skills]
    safe_skills.append("Python")
    print("  手工复制后原结构:", original_profile)
    print("  手工复制后的新结构:", safe_profile)
    print("  内层身份不同:", original_profile[1] is not safe_profile[1])

    print("练习 3：成绩排名")
    students: list[tuple[str, int]] = [("Alice", 88), ("Bob", 95), ("Cara", 91)]
    ranking: list[tuple[str, int]] = sorted(students, key=lambda student: student[1], reverse=True)
    champion: tuple[str, int], *other_students = ranking
    print("  原列表:", students)
    print("  新排名:", ranking)
    print("  冠军/其余:", champion, other_students)
```

```
209 def demonstrate_readme_exercises() -> None:
246     print("练习 4: 整理标签文本")
247     raw_tags = "  java,python,sql  "
248     stripped_tags: LiteralString = raw_tags.strip()
249     split_tags: list[LiteralString] = stripped_tags.split(",")
250     upper_tags: list[LiteralString] = stripped_tags.upper().split(",")
251     joined_tags: LiteralString = " / ".join(upper_tags)
252     print("  原始文本:", repr(raw_tags))
253     print("  strip 后:", repr(stripped_tags))
254     print("  split 后:", split_tags)
255     print("  upper 后:", upper_tags)
256     print("  join 后:", joined_tags)
257
258
259 def main() -> None:
260     demonstrate_indexing()
261     demonstrate_slicing_and_copying()
262     demonstrate_list_mutation()
263     demonstrate_new_object_or_in_place()
264     demonstrate_repeated_nested_lists()
265     demonstrate_tuples()
266     demonstrate_strings()
267     demonstrate_readme_exercises()
268
269
270 if __name__ == "__main__":
271     main()
272
```